

Investigating UML- and Ontology-Based Approaches for Process Improvement in Developing Agile Multi-Agent Systems

Thomas Moser, Klemens Kunz, Kamil Matousek, Dindin Wahyudin
Institute for Software Technology and Interactive Systems
Vienna University of Technology, Vienna, Austria
{TMoser, K.Kunz, Matousek, Dindin}@ifs.tuwien.ac.at

Abstract

Multi-agent systems (MAS) that link business and physical domains such as in production automation need to be reconfigured correctly and efficiently to adapt to new requirements. The standard UML-based approach only partly supports the reconfiguration process to capture agent classes and their instances, while ontologies allow modeling all aspects of MAS design in one common continuous model.

In this paper we introduce a MAS development life cycle and focus on the product-specific reconfiguration of a system built mostly from reusable agents. We investigate the process variants based on (a) UML and (b) ontologies. We evaluate both process variants in a feasibility study using fundamental illustrative scenarios from an industrial production automation environment and derive lessons learned for process improvement in building sustainable MAS in the scope of production automation.

Keywords: Process modeling and improvement, Software reuse and product lines, Agile Multi Agent System Development.

1. Introduction

Global competition and e-commerce opportunities challenge many organizations to adapt their systems to rapid and unexpected changes of requirements. Systems for production automation, which link classical software information systems to systems that contain hardware elements, undergo a particularly challenging evolution [9]: Mechanical devices and electronic devices have increasingly more storage, faster computation and communication capabilities, which add to their potential utility with new degrees of freedom for design but also add to the complexity of system development and evolution [15].

Multi-agent systems (MAS) design [10, 11] can help encapsulate the complexity [1] by providing appropriate object views on the system parts, which can be supported with standard UML diagrams [7].

However, a major challenge is to support hardware and software reconfiguration processes with consistent models that minimize the risk of introducing and not finding defects during the reconfiguration process as UML seems to provide only weak support for designing, changing, and verifying configurations in a way that allows efficient and

correct adaptation to new requirements, new behavior, and new support technology. Due to requirements changes, software architects and developers want to easily reconfigure the operational system while quality assurance (QA) personnel need to measure the system performance, identify defects and the potential impact of a defect [12]. The current standard development approach using UML notation only partly supports the reconfiguration process to capture agent classes, their instances, and the configuration.

Recently the development of and improving tool support for ontologies for software engineering promises to effectively and efficiently incorporate all relevant MAS configuration models in one modeling space, providing less fractured and better integrated views [8] both for designers and QA personnel to better prevent and detect defects early during reconfiguration.

In this paper, we describe an ontology-based approach for the MAS development process that can provide continuous model views as needed for MAS reconfiguration in order to capture both classes and instances, even dynamic instances. We propose a concept to compare the benefits and limitations of ontologies to conventional UML-based development in the context of relevant MAS design and reconfiguration scenarios. For an initial empirical evaluation, we suggest a set of development complexity and quality criteria and apply the concept to scenarios derived from the industry-relevant production automation information system domain. The remaining part of this paper is organized as follows: Section 2 summarizes related work on agile MAS development. Section 3 motivates research issues and the application scenario. Section 4 outlines the UML- and ontology-based development approaches for MAS reconfiguration to compare these approaches. In section 5 we provide an initial evaluation of the concept and discuss the results. Section 6 concludes and suggests future research.

2. Related Work

Today software systems that support production automation face two major challenges: (a) fault-tolerant (safe) production systems grow more complex (e.g., larger systems, technology for more fine-grained control of production elements, more degrees of freedom of the elements

involved) and thus are harder to model and verify for consistency and (b) the systems have to deal with changes in their environment (e.g., failures, changes in configuration) in more sustainable and more responsive (i.e., agile) ways [10]. This section summarizes related work on these challenges for MAS development processes in the production automation domain.

2.1. Reconfigurable Multi-Agent Systems

Studies of mass production automation systems [4, 9] conclude that the design of modern systems should be proficient and responsive enough to efficiently manufacture customized products for high-volume markets. This goal needs effective and efficient reconfiguration processes on the production floor, i.e., capabilities for stepwise expandability, integration of new components, and dynamic plan layout [14]. Multi-agent system (MAS) design can help address these goals on the software level [13]. These agents can model the states, behavior of and interfaces between actual entities in the system (like hardware, machines, and software systems) in a natural and flexible way.

While the idea of agents [14, 15] helps structuring a MAS, the flexible reconfiguration and design of the communication among agents remains a challenge as the system should react well to certain classes of failures, which are partly considered by MAS design and development methods [13, 16, 18].

2.2. Multi-Agent System Design for Reconfiguration

For creating a MAS one has to address aspects on two levels (similar to a software product line process): (1) domain level: agent identification, data definition, behavior definition, coordination and communication definition [10] and (2) product level: iterative reconfiguration of system versions consisting of domain-level agents.

New requirements often need design reconfigurations such as adding new agents, replacing agents or modifying the capability or cooperation of agents within the system. However, complexity aspects of a large system, such as communication paths between agents and the number of components to be handled, make the task to design the reconfiguration of the agents and layout error prone. Therefore MAS design methods need to provide capabilities such as reusability of design components and knowledge, reconfiguration during run time, and incorporation of tool-supported QA.

Current approaches to agent system design such as Gaia [18], Massive [7], Agent UML [13] and Tropos [11] are mostly goal driven: an agent system is designed by iteratively decomposing system goals until they can be assigned to individual agents. However, goal-driven design goal decomposition may lead developers to work out solutions to common design problems without benefiting from

past knowledge [16] resulting in duplicate effort, inconsistent design, and poor traceability of the system design.

One approach to provide reusable solutions to recurring MAS design problems is pattern-driven design [1, 3, 16]. In a pattern-driven approach [16] agent systems are developed starting from recurring design solutions and making them available as (domain-specific) patterns, later the designer can select appropriate patterns for their specific system application derived from a given set of patterns and pattern languages. In the production automation domain, patterns can describe agent roles, agent coordination, work orders, system failures and recovery activities. Some tasks can be supported by tools such as pattern documentation, requirement based selection, and patterns modeling tools. However, built-in support for the reconfiguration of an evolving system based on agent patterns is still not well understood.

2.3. Approaches to Multi-Agent System Design

The decomposition and transformation of the system from design to agent system implementation typically uses models in UML notation. In this UML-based approach descriptions of system such as agent identification, data definition, behavior definition, coordination and communication definition are captured and visualized in UML diagrams (e.g. use cases, class diagram and behavior diagrams). QA is performed by reviewing the UML models before implementation [2], test case generation derived from UML diagrams, according to model coverage criteria [12]. However, UML diagrams seem not well suited to capture relevant aspects of system reconfiguration such as agent class instances and the configuration of class instances. Thus, in an UML-based approach, the configuration of agents has to be represented with the aid of “external” models such as an XML configuration file, since the configuration parameters need to be presented to the agents in a machine-readable way. In case of reconfiguration, the XML configuration file regularly has to be changed and reviewed for internal and external consistency. To find all dependencies and modifications necessary for a particular reconfiguration task is often error prone and can take considerable effort.

To address these challenging issues, in this paper we explore as alternative the use of ontologies to capture agent patterns and configuration information as input for the MAS reconfiguration process. Currently, ontologies are used in MAS design to capture the agent communications [6, 8] and for checking the consistency of UML models [5]. Ontologies allow storing the schema and data together within a continuous model, preventing fractured views on several models and effectively supporting checks for inconsistency and incompleteness regarding all involved models. Further, ontologies can be used both during design time and run time, which allows using the design information at run time and also feeding back run time changes to

the next cycle of design in an agile integrated and iterative process.

3. Research Issues & Application Scenario

Our research goal is to compare the properties of an ontology-based process for reconfiguration of MAS production systems to the properties of a traditional UML-based process to better understand the strengths and limitations of these alternatives, provide guidelines for MAS designers, and identify issues for further work. From the generic MAS reconfiguration process [19] we derive (a) a process variant based on a traditional UML-based modeling approach (with extensions that cover apparent limitations of UML) and (b) an ontology-based approach that promises to model important reconfiguration aspects in a continuous model. New development approaches for MAS design, such as the proposed ontology-based approach, are expected to bring benefits to software development like faster or more efficient development. To identify general properties of models selection, we worked with experts in software development in the product automation domain. Later we derived the following criteria to compare the alternative approaches.

- *Model complexity.* For a given reconfiguration scenario we measure complexity as the number of elements/relationships that need to be developed or modified. A more complex system model is typically harder to handle for the system developer and designer, e.g., for the integration of the model elements, and traceability of the model elements along the life cycle as the system developer and designer has to find all dependencies and modifications necessary for a particular reconfiguration.
- *Modeling effort.* In a scenario we measure the effort for the steps by determining the average effort for tasks that have to be applied to a certain number of elements derived from the scenario.
- *Quality risk.* We evaluate the risk level of introducing or not identifying relevant and typical reconfiguration defects such as inconsistencies and incorrectness of models as on a Likert scale as low, medium, or high. Part of the risk assessment is the built-in support of an approach for QA that helps preventing or early identifying relevant classes of defects.

The research application scenario in production automation is a typical MAS application from the industrial manufacturing domain [6]. Figure 1 shows a typical workshop layout where products are assembled from a number of product parts. The system consists of agents and components (group of agents) such as storage areas, where parts (raw materials, semi-finished and finished products) are stored; assembly areas, where robots assemble the parts into products; and a finishing area, where the products are taken out of the system and delivered to another storage area or to a system for further development and shipment to the customer. The product parts and finished products

are transported on pallets using conveyor belts. Intersections connect conveyor belts, a diverter is an intersection with one incoming and two outgoing conveyors; a junction connects two incoming to one outgoing conveyor.

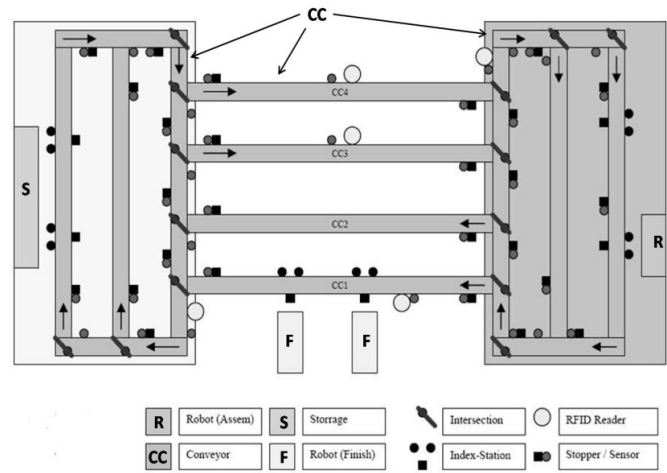


Figure 1. Production automation scenario [6].

Note that key parts of the system can be redundant to increase system throughput capabilities and allow keeping the system running during maintenance/reconfiguration phases of a redundant system part. A key aspect is that system coordination can dynamically react to changes in resource availability and changes in system capabilities.

4. Reconfiguration with UML & Ontology

MAS reconfiguration is an essential capability for production automation applications which need to be adjusted for maintenance and evolution, e.g., new work layouts or new technologies. Thus, this capability should be supported by repeatable processes that can be monitored, audited, and improved over time.

In this section we refer to a generic MAS life cycle [19] that includes a reconfiguration process, and derive process variants for UML-based and ontology-based reconfiguration. The generic MAS life cycle distinguishes between the domain level and production line level: The domain level represents the development activities for a reusable set of agents that can be used for a wide range of application scenarios. The production line level outlines the activities of actual system reconfiguration: reconfiguration analysis, reconfiguration implementation, and testing and simulation of new configurations.

4.1. UML-Based Reconfiguration Process

From the generic process in [19] we derive the UML variant in Fig. 2a.

Step 1 Reconfiguration Analysis (1a) and Design (1b). The UML-based approach starts from the stakeholder

requirements input with step 1a, reconfiguration analysis, resulting in a Use Case Model, which depicts the system functionalities as use cases and system failures as failure cases. This step concludes with QA of the use case model against the original requirements.

In step 1b, system design, the designer identifies from this global system functionality model the agents needed to provide the functionality. The distinction of the reconfiguration analysis and design step of the generic life cycle into two separate steps with additional QA cycles is expected to provide better quality of the final system, as it is possible to check the use case model against the reconfiguration requirements at an early stage of the life cycle, thus minimizing the risk of missing requirements in the final system.

Each agent is designed using a variety of UML models, depending on which aspect of the agent should be modeled. For structural design class diagrams show basic parameters of an agent, like an identifying name and interfaces. Nevertheless, this method does not provide concrete insight into the agent class instances and the number of agent connections that makes sense in various situations.

Step 2 Reconfiguration Implementation. From the UML models parts of the implementation can be generated automatically, like the basic structure of agent classes. The behaviour of the agents can partly be derived from the agent tool box, which provides reusable agents, with minor configuration-dependent modification; product-specific behaviour has to be implemented manually, although it is possible to transform some behaviour from state charts and sequence diagrams into code (e.g. failover behavior derived from sequence charts). The implementation covers agent classes, not instances of agents, as the number of instances, configuration, and specific behaviour parameters are defined in the XML configuration file.

Step 3 Reconfiguration Testing and Simulation. For safety-critical systems, it is necessary to measure system quality and performance (e.g., remaining unresolved (un-critical) defects, response to failure scenarios, and service levels; system throughput in different workload scenarios) of the new configuration. One approach is to perform lab monitoring using a simulation tool that represents relevant properties of the target system [10]. The successfully tested configuration can be deployed to the target environment and be used as new current system configuration for step 1. Defects found during simulation and testing are reported to design or implementation as appropriate. The UML-based approach ultimately results in the XML configuration file, which can be tested with a simulation tool. UML diagrams support testing of the configuration, as we can derive system test cases from the use case model and sequence charts; and agent unit test cases from state charts and sequence charts.

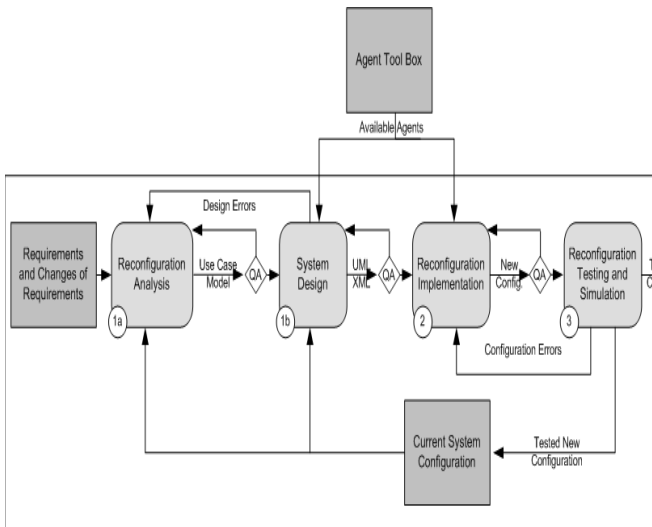


Figure 2a. UML Rconfiguration LifeCycle with QA.

Modeling of the overall system behavior is more difficult with UML diagrams, especially when dealing with a large number of agents. Thus a complete system sequence diagram is often large and surprisingly hard to handle and understand. Furthermore, UML does not provide methods to define the detailed configuration of agent instances.

In the UML reconfiguration process typical defects are: inconsistencies between models and/or violated dependency relationships if a change is not consistently applied to all models, use case variants may be forgotten or interaction effects not well understood. The QA cycles mainly consist of manually reviewing the models against the requirements and against model aspects that need to stay consistent. External model checkers exist, but are often not well-integrated elements in the UML methodology.

4.2. Ontology-based Reconfiguration Process

From the generic process in [19] we derive the ontology process variant in Fig. 2b.

Step 1 Reconfiguration Analysis and Design. Reconfiguration requirements form the input of the reconfiguration domain analysis step, which is composed of the tasks concept and relationship identification as well as use case analysis of ontology and agents. Domain test cases are created as additional output to be used in latter QA steps.

System design is the next step, where agent toolbox concepts reusing other more general upper-level ontologies are taken as input, where both static and dynamic structures of the system ontology and software agents are designed. The ontology contains static information on the production line components (called resources such as robots or pallets) and their structure, possible failure types, as well as some behavioural aspects like work orders derived from business orders of customers, agents and resource status modeled by dynamically created and updated instances. Rules represent the reactive event processing components, which enable to directly manipulate ontology components and instances as well as to communicate with software agents via sending messages.

The QA step after system design is used to ensure formal consistency and validity of the design models by performing type checks and semantic validation. Requirement satisfaction by the design is evaluated; expert reviews of the ontology structure are performed to identify and resolve inconsistencies. Further, the reasoning capability of ontologies allows expressing and enacting routine checks automatically on classes and instances. Issues and defects are reported back to reconfiguration analysis and design for resolution.

Step 2 Reconfiguration Implementation. Implementation follows the system design, where the designed ontology can be directly used as input and is in this step extended by concrete instances and their property values. In a production automation environment additional programming can integrate the agent framework (e.g. JADE [8]), communication (e.g., FIPA ACL language [15]) and logging capabilities.

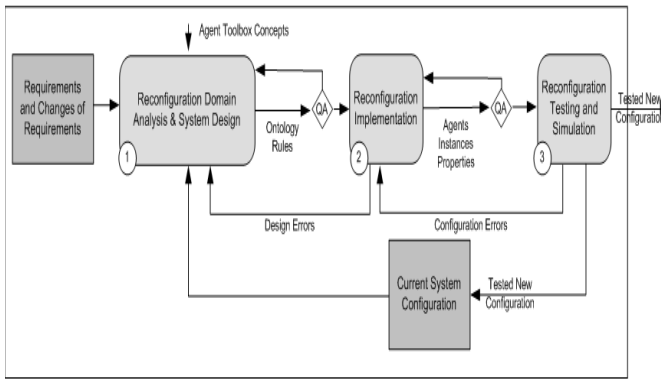


Figure 2b. Ontology-based Reconfiguration Life Cycle with QA.

For QA the new configuration has to pass a static validity test, i.e. checking whether each agent has a consistent interface (input and output) with other agents or components. After implementation, software unit testing using domain and other test cases is performed. The reasoning capability of the ontology can be used to check the consistency of the configuration both of classes and instances including their dependencies to other components and design patterns. This integrated capability helps to take away routine effort from QA personnel and also designers and implementers. Issues could be design defects that get reported to step 1, or configuration defects that feed back directly into the implementation step.

Step 3 Reconfiguration Testing and Simulation. The last step before deployment is system operation in a testing and simulation environment, where normal operation using inter-agent communication via messages is functional, ontology instances and property values are being dynamically asserted and removed, and logical rules are being fired, when appropriate. Moreover, hardware failures can be simulated, monitored and possibly recovered with use of ontology and rules at run-time. Thus with an ontology

approach similar model views can be used during design, implementation, and run time helping to ensure consistent models as input to the next reconfiguration cycle.

5. Evaluation Study and Discussion

This section reports on an initial evaluation of the process variants described in Section 4 based on a scenario taken from industry and discusses the results. We focus on the reconfiguration level process and investigate the enactment of the general process from the roles MAS developer and QA (a) using a traditional UML-based modeling approach (see Section 4.1) and (b) using an ontology-based approach that promises to model important reconfiguration aspects in a continuous model (see Section 4.2). We conduct a feasibility study to investigate actual strengths and limitations of both approaches. In the study the roles were taken by 2 persons with good technical experience in the application area and UML/ontologies fitting to the process variant they were to enact.

In this work we also identify typical defects that may occur in UML-based approach that should be addressed by the ontology-based approach.

The subsections describe the scenarios and the results of modeling new configurations in the context of the system depicted in Figure 1 using both approaches. The research application context involved 7 agent classes with the following number of agent instances (see Figure 1): 25 conveyors, 6 junctions; 6 diverters, 6 index stations; 4 RFID readers, 3 robots, and one storage rack. We analyzed both approaches regarding model complexity; modeling effort and quality risk (see Section 3).

Scenario 1 – adding a conveyor: In order to increase the throughput of the system a new connection conveyor should be added, leading from the storage area to the assembly area (redundant conveyor parallel to CC3 in Figure 1).

Scenario 2 – conveyor removal: One connection conveyor should be removed from the system (e.g., CC3 in Figure 1).

Scenario 3 – change of conveyor direction: The direction of a connection conveyor should be changed (e.g., CC4 in Figure 1).

Precondition for each of the reconfiguration scenarios is to conduct a dependency analysis to identify relevant dependencies between agent instances. In this context, dependencies exist with the direct neighborhood of an agent (e.g., in- and out-node of a conveyor), but also in design patterns where a group of agents cooperates (e.g., several conveyors can provide a loop to temporarily store pallets). Dependencies impact design changes and QA in both approaches similarly.

5.1. Scenario - Change of a Resource Agent - UML

This subsection describes the results of reconfiguration using the UML-based approach. In order to identify, if adding, removing, and change direction of the conveyor has an impact on the system behaviour, we conduct a dependency analysis to find out, which behaviour or which design patterns may be affected. This dependency analysis has to be conducted manually by analyzing existing design patterns.

Just to give an example, adding a conveyor to the system results in a small number of changes. The first step is to analyze the requirements for the new conveyor. If the conveyor does not need new functionality beyond the capabilities available from the agent tool box, we can take the agent as it is. In this case the only change is in the XML configuration file, which has to be extended by the new conveyor.

However, if the conveyor is added as an additional rerouting possibility because of higher work load in the system, it is necessary to model the rerouting design pattern with additional sequence charts. It is also important to model the failure handling behaviour for the new conveyor with additional sequence diagrams. Also a new design pattern may be necessary. The changes made to the different models and the newly created models result in at least one QA cycle (manual review) for each of the models.

5.2. Change of a Resource Agent - Ontology

This subsection describes the results of reconfiguration using the ontology approach.

Scenario 1 – adding a conveyor: The requirement to include a new conveyor in the system can be satisfied by creating a conveyor-supervising software agent, which is responsible for the proper behaviour and for scheduling of the underlying hardware conveyor. Further we add a new instance of *Conveyor* concept in the ontology and fill in its properties. Defects introduced during adding activities, e.g., a non-connected conveyor, can be prevented or found early by running the ontology reasoner with logical queries based on the system design and coordination patterns that are applicable to a conveyor.

Scenario 2 – conveyor removal: In order to remove a conveyor from the system, the responsible software agent has to be destroyed, the instance of *Conveyor* concept, as well as the supervising agent instance, have to be removed from the ontology and all the machine agents supervising connected resources (e.g., intersections) have to delete the link to conveyor instances in the relevant ontology property values. Defects introduced during the activities, e.g., a non-connected intersection or separated line clusters (partially connected network) can be again prevented by running logical queries. Some of these situations can be resolved by changing the direction of a remaining conveyor.

Scenario 3 – change of the conveyor direction: Changing the conveyor direction requires few actions to be per-

formed: The values of the conveyor properties *inNode* and *outNode* have to be exchanged and connected intersection agents have to change the intersection instance class from *Diverter* to *Junction* and vice-versa and shift the changed conveyor instance between its *inConveyor* and *outConveyor* properties. Changing the conveyor direction can however cause defect situations when, e.g., the junction (i.e., new in-node) does not have the switching capability, so it cannot act as a diverter [19], or the direction change could cause the creation of a logistically unreachable subsystem. After changing the hardware conveyor direction, all the software agents controlling diverter intersections perform recalculation of the shortest paths to the line nodes by running corresponding general logical rules. These rules are able to discover any unreachable nodes and issue warnings.

Logical reasoning, on the other hand, can effectively prevent the logical changing of the junctions to diverters as their switching behaviour is required and has to be denoted in the corresponding property value. Some of such defect situations might have a solution in changing the direction of another conveyor in the system.

5.3. Comparison of UML- & Ontology Processes

Table 1 provides a side-by-side view on complexities, effort, and risks encountered in the study scenarios (see [19] for a more detailed discussion). Process effort measurements depend on several factors in addition to process and tool support (e.g., task selection and skill factors, maturation of subjects), thus we present the effort ranges without consideration of statistical significance. We are well aware of the external validity limitations of such an initial study; however, such studies are important to motivate the investment into more comprehensive empirical studies [17] in practitioner environments (for a more detailed comparison see [19]).

Overall both approaches were well suited to deal with the limited complexity of the initial evaluation scenarios. The main difference in evaluation between the two approaches is in the modeling effort, which is considerably higher for the UML-based approach than for the ontology-based approach, which can be supported by automated reasoning. Manual dependency checks and QA need considerably more time and are also more error prone. A similarity in the approaches is in the interdependency of effort for conducting the actual model changes and the number of models to be changed. However, in the UML-based approach we have to change more artifacts than in the ontology-based approach.

The quality risks of the UML-based approach depend mainly on the accuracy of the system designer. Especially the dependency analysis and QA tasks have to be conducted carefully and completely to avoid mistakes, which may lead to system failures, which are only detected during system operation. Thus, the quality risk is rated medium. The quality risk for the ontology-based approach is

rated low, because QA can be well supported with automated reasoning, which avoids the human factor and thus has a very low probability of missing inconsistencies.

Table 1: Comparison of UML-and Ontology-based approaches (times in minutes).

	UML-Based Approach	Ontology-based Approach
Scenario 1 – adding a conveyor		
Model complexity	Number of affected artifacts: 6	Number of affected artifacts: 4
Modeling effort	Model changes: 50-60 min. Dependency analysis: 80-90 min. Quality assurance: 160-180 min.	Model changes: 35-45 minutes Dependency analysis: 50-60 min. Quality assurance: 25- 35 min.
Quality risk	medium	low
Scenario 2 –conveyor removal		
Model complexity	Number of affected artifacts: 7	Number of affected artifacts: 4
Modeling effort	Model changes: 60-70 min. Dependency analysis: 80-90 min. Quality assurance: 160-180 min.	Model changes: 25-35 min. Dependency analysis: 50-60 min. Quality assurance: 25-35 min.
Quality risk	medium	low
Scenario 3 –change of conveyor direction		
Model complexity	Number of affected artifacts: 10	Number of affected artifacts: 3
Modeling effort	Model changes: 90-100 min. Dependency analysis: 80-90 min. Quality assurance: 160-180 min.	Model changes: 25-35 min. Dependency analysis: 50-60 min. Quality assurance: 25-35 min.
Quality risk	medium	low

Perceived strengths of the UML-based approach for reconfiguration were to help designers get an overview on a domain and identify components to work with: a) to provide well readable and understandable visualization for the designer and developer, especially for the software development process of reusable assets (e.g., an agent tool box); and b) to provide a well understandable high-level overview on agent classes, their properties and of agents states, so behavior modeling can be focused to most relevant agents.

Perceived weaknesses of UML-based approach for reconfiguration were a) to needs model extensions to the reconfiguration information, which leads to a fractured view (that needs to be compensated with appropriate QA methods, e.g., inspection of consistency between several models that capture partly overlapping aspects); b) could not well capture the agent instance and configuration information; c) had a higher error proneness due to the need for manual model reviews.

The ontology-based approach seems to be stronger to keep an overview on all the detailed information and dependencies on agents and instances over several iterations of reconfigurations in the context of a well-defined big picture.

Perceived strengths of ontology-based approach for reconfiguration were a) to capture system configuration description (schema and data) in an integrated model; b) the integrated model allows tool support for QA checks

(reasoning to conduct consistency and plausibility checks among the model aspects); c) to provide the source to explore the system structure using an ontology editor; and d) to provide a model that is directly usable both at design time and run time, enabling dynamic reconfiguration checks even at run-time (e.g., when handling of hardware failure scenarios).

Perceived weaknesses of ontology-based approach for reconfiguration were a) higher complexity of the ontology model; b) to provide human readable visualization of relationships among entities and a general overview on a domain with generic standard tools; c) the challenge to understand the contribution of an ontology element without good understanding of the domain, the design task, and the overall ontology design. In addition it has to be stated that compared to traditional SE qualifications like data modeling using UML, ontologies are a fairly new topic. Therefore, additional training or qualification of the involved software engineers is needed in order to allow the ontology-based approach to be used in a efficient way.

6. Conclusion and Future Work

Multi-agent systems (MAS) in production automation need to be reconfigured correctly and efficiently to adapt to new requirements, new behavior, and new support technology. In this paper we focused on the reconfiguration process in the MAS development life cycle and investigated the enactment of the general process as (a) an UML-based approach with XML configuration extensions and (b) an ontology-based approach that models important reconfiguration aspects in a continuous model. For initial empirical evaluation, we applied the concept in fundamental scenarios derived from the production automation information system domain, and investigated complexity, effort, and quality risk for both approaches.

Major results are: 1) The current standard development approach using UML notation only partly supports capturing agent classes, the configuration and their instances. Modeling the interaction of instances (e.g., in design patterns) with sequence diagrams can lead to a large number of diagrams even for relatively simple scenarios. UML diagrams are easy to follow and dependencies in UML diagrams can be well deduced. However, keeping several types of UML diagrams synchronized with each other and the actual system configuration can take considerable effort. 2) Ontologies allow modeling all aspects of MAS design in a common model, but contain considerably more entities and relationships than presented in UML diagrams modeling a similar scenario (as instances and their dependencies are also modeled adding to the ontology complexity). Ontologies are essentially logical text-based constructs that can model many dimensions of relationships among element, thus a generic visualization is a major challenge, while logical reasoning can be well supported for dependency analysis and consistency checking. Ontologies can

directly contain the system configuration, lowering the synchronization risk.

Overall both approaches were well suited to deal with the limited complexity of the evaluation scenarios. We found the UML-based approach particularly well suited to help designers get an overview on a domain and identify components to work with. The ontology-based approach seems to be stronger to keep an overview on all the detailed information and dependencies on agents and instances over several iterations of reconfigurations in the context of a well-defined big picture.

For further work we see much promise in a systems design process combining the strengths of both UML and ontologies beyond the reconfiguration process described in this paper. The next steps in research should pursue the following directions:

- Prototype a design process that uses UML to provide domain context and overview on system class level; ontologies to map design patterns to instances and tool support for QA; based on the success factors identified in the evaluation study, e.g., domain-specific visualization for ontologies.
- Propose methods and tool support for more effective and efficient dependency analysis and QA to mitigate the risk of defects and their impact on system quality and rework effort, e.g., test case generation from design patterns and instance information in the ontology and visualization support for QA.
- Investigate the use of ontologies at run time to support checking the safety of agent instance states for more effective failure handling to further enhance agility and sustainability of the application system. Feeding back run time changes to the next cycle of design helps ensure information consistency in an agile process.
- Propose the use of ontologies for providing a continuous data model during the life cycle of a system. Since ontologies are a fairly new topic in SE, traditional data modeling methods like UML need to be extended by ontologies as a first step in order to guarantee a higher acceptance rate of this new data modeling method.

The software system level of production automation provides an industry-relevant application context for research work and allows the evaluation of approaches that can be readily transferred to comparable mission-critical system domains where effective and efficient reconfiguration is an important success factor.

References

- [1] Aridor, Y., Lange, D., Agent Design Patterns: Elements of Agent Application Design, Second Intl. Conference on Autonomous Agents, IEEE, 1998
- [2] Biffl, S., Mordinyi, R., Schatten, A.: A Model-Driven Architecture Approach Using Explicit Stakeholder Quality Requirement Models for Building Dependable Information Systems, in Proceedings of the 5th International Workshop on Software Quality, International Conference on Software Engineering (ICSE2007), 2007
- [3] Deugo, D., Weiss, M., and Kendall, L., Reusable Patterns for Agent Coordination, in: Omicini, A., et al (eds.), Coordination of Internet Agents, Springer, 2001
- [4] Felfernig, A.: Standardized Configuration Knowledge Representations as Technological Foundation for Mass Customization, IEEE Transaction on Engineering Management on; 2004
- [5] Guizzardi, G., Wagner, G., Guarino, N., van Sinderen, M.: An Ontologically Well-Founded Profile for UML Conceptual Models, in Proc. of 16th International Conference on Advanced Information Systems Engineering (CAISE 2004), Springer LNCS, 2004
- [6] Koppensteiner, G., Konzeption und Entwicklung einer Ontologie für ein wissensbasiertes Automatisierungssystem, Master Thesis, Vienna University of Technology, 2007.
- [7] Lind, J., Iterative Software Engineering for Multi-Agent Systems: The MASSIVE Method, 2001
- [8] Merdan, M.; Kordic, V.; Zoitl, A.; Lazinic, A.; Knowledge-based Multi-agent Architecture, International Conference on Computational Intelligence for Modelling, Control and Automation, (CIMCA 06) p. 55, Sydney, Australia; 2006.
- [9] Mikkola, J. H.: Management of Product Architecture Modularity for Mass Customization: Modeling and Theoretical Considerations, IEEE Transaction on Engineering Management on Mass Customization Manufacturing Systems vol 54 no. 1, pp 57-69, 2007
- [10] Marik, V., Vrba, P., Hall, K., Maturana, F.: Rockwell Automation agents for Manufacturing, Int. Conf. on Autonomous Agents, in Proc. 4th int. joint Conf. on Autonomous Agents and Multiagent Systems 2005.
- [11] Paolo Bresciani. Paolo Giorgini. Fausto Giunchiglia. John Mylopoulos. Anna Perini. TROPIS: An Agent-Oriented Software Development Methodology. In Journal of Autonomous Agents and Multi-Agent Systems. Kluwer Academic Publishers. May 2004
- [12] Reuys, A., Kamsties, E., Pohl, K., Reis, S.: Model-Based System Testing of Software Product Families, In Proceeding of the 17th International Conference on Advanced Information Systems Engineering (CAISE 2005), Springer LNCS, 2005
- [13] Sellers, B., H, Giorgini, P.: Agent-Oriented Methodologies, Idea Group Publishing, 2005
- [14] Van Leeuwen, E.H., and Norrie, D.: Intelligent Manufacturing: Holons and Holarchies, Manufacturing Engineer, vol. 76, no. 2, 1997, pp. 86-88.
- [15] Vrba, P., MAST: Manufacturing Agent Simulation Tool, Emerging Technologies and Factory Automation, 2003. Proceedings. ETFA apos;03. IEEE Conference Volume 1, Issue , pp. 16-19 Sept. 2003
- [16] Weiss, M.: Pattern-Driven Design of Agent Systems: Approach and Case Study, In Proceeding of CAiSE 2003, Springer-Verlag Berlin LNCS 2681, pp. 711-723, 2003
- [17] Wohlin, C., Runeson, P., Höst, M., Ohlsson, M., Regnell, B., Wesslén, A.: Experimentation in software engineering: an introduction, Kluwer Academic Publishers, Norwell, MA, 2000
- [18] Wooldridge, M., Jennings, N., and Kinny, D., The Gaia Methodology for Agent-oriented Analysis and Design, Journal of Autonomous Agents and Multi-Agent Systems, 2002
- [19] Technical Report "Investigating UML- and Ontology-Based Approaches for Developing Agile Multi-Agent Systems" available at: http://qse.ifs.tuwien.ac.at/publication/IFS-QSE-0712_UML+Ontology_for_MAS_Reconfiguration.pdf